





MiniPoly: Automatic Extraction of Efficient JavaScript Polyfill from Language Specification

Jihyeok Park



2026.10.08 @ UC Berkeley EECS

Programming Language Research Group

PLRG

- Research Field - **Programming Languages** and **Software Engineering**
  - Program Analysis
  - Mechanized Language Specification
  - Automated Testing
  - Program Synthesis & Repair
- **Recent Publications** (2020~)
  - **PLDI** (2023, 2024) / **SAS** (2026)
  - **ICSE** (2021) /
    FSE (2021, 2022, 2025-Demo) /
    ASE (2020, 2021, 2025, 2026)
  - **CACM** (2024) / **CSUR** (2021) /
    TOSEM (2026)



Built-in APIs in JavaScript

ES2025

Native Code
(e.g., C++, Java)

```
const a = new Set([1, 2, 3]);
const b = new Set([1, 2]);
a.isSupersetOf(b);   // true
```



≤ 126

✗



≤ 2025.09

✗



≥ 122

✓



≤ 16.7

✗



≤ 21

✗

Polyfill

- **Polyfill** is a **JavaScript** implementation of built-in APIs
- **core-js** is used by **41% of websites** and **≥ 46M downloads per week**

```
module.exports = function isSupersetOf(other) {
  var O = aSet(this);
  var otherRec = getSetRecord(other);
  if (size(O) < otherRec.size) return false;
  var iterator = otherRec.getIterator();
  return iterateSimple(iterator, function (e) {
    if (!has(O, e)) {
      return iteratorClose(iterator, 'normal', false);
    }
  }) !== false;
};
```

core-js

Polyfill

```js
import 'core-js/actual/set/is-superset-of';
const a = new Set([1, 2, 3]);
const b = new Set([1, 2]);
a.isSupersetOf(b);   // true
```

```js
module.exports = function isSupersetOf(other) {
  var O = aSet(this);
  var otherRec = getSetRecord(other);
  if (size(O) < otherRec.size) return false;
  var iterator = otherRec.getIterator();
  return iterateSimple(iterator, function (e) {
    if (!has(O, e)) {
      return iteratorClose(iterator, 'normal', false);
    }
  }) !== false;
};
```

core-js



≤ 126



≤ 2025.09



≥ 122



≤ 16.7



≤ 21











ECMA-262 - JavaScript Language Specification

Built-in

Set.prototype.isSupersetOf (*other*)
1. Let *O* be the **this** value.
2. Perform ? RequireInternalSlot(*O*, [[SetData]]).
3. Let *otherRec* be ? GetSetRecord(*other*).
4. If SetDataSize(*O*.[[SetData]]) < otherRec.[[Size]], return **false**.
5. Let *keysIter* be ? GetIteratorFromMethod(*otherRec*.[[SetObject]], *otherRec*.[[Keys]]).
6. Let *next* be NOT-STARTED.
7. Repeat, while *next* is not DONE,
   a. Set next to ? IteratorStepValue(*keysIter*).
   b. If *next* is not DONE, then
      i. If SetDataHas(*O*.[[SetData]], *next*) is **false**, then
         1. Perform ? IteratorClose(*keysIter*, NormalCompletion(UNUSED)).
         2. Return **false**.
8. Return **true**.

tc39/ecma262

Status, process, and documents for ECMA-262

TC 39

Syntax

AdditiveExpression[Yield, Await] :
MultiplicativeExpression[?Yield, ?Await]
AdditiveExpression[?Yield, ?Await] + *MultiplicativeExpression*[?Yield, ?Await]
AdditiveExpression[?Yield, ?Await] - *MultiplicativeExpression*[?Yield, ?Await]

AdditiveExpression : *AdditiveExpression* + *MultiplicativeExpression*

1. Return ? EvaluateStringOrNumericBinaryExpression(*AdditiveExpression*, +, *MultiplicativeExpression*).

Semantics

Current Approach - **Manual Translation**

Built-in

Set.prototype.isSupersetOf (*other*)

1. Let *O* be the **this** value.
2. Perform ? RequireInternalSlot(*O*, [[SetData]]).
3. Let *otherRec* be ? GetSetRecord(*other*).
4. If SetDataSize(*O*.[[SetData]]) < otherRec.[[Size]], return **false**.
5. Let *keysIter* be ? GetIteratorFromMethod(*otherRec*.[[SetObject]], *otherRec*.[[Keys]]).
6. Let *next* be NOT-STARTED.
7. Repeat, while *next* is not DONE,
   a. Set next to ? IteratorStepValue(*keysIter*).
   b. If *next* is not DONE, then
      i. If SetDataHas(*O*.[[SetData]], *next*) is **false**, then
         1. Perform ? IteratorClose(*keysIter*, NormalCompletion(UNUSED)).
         2. Return **false**.
8. Return **true**.

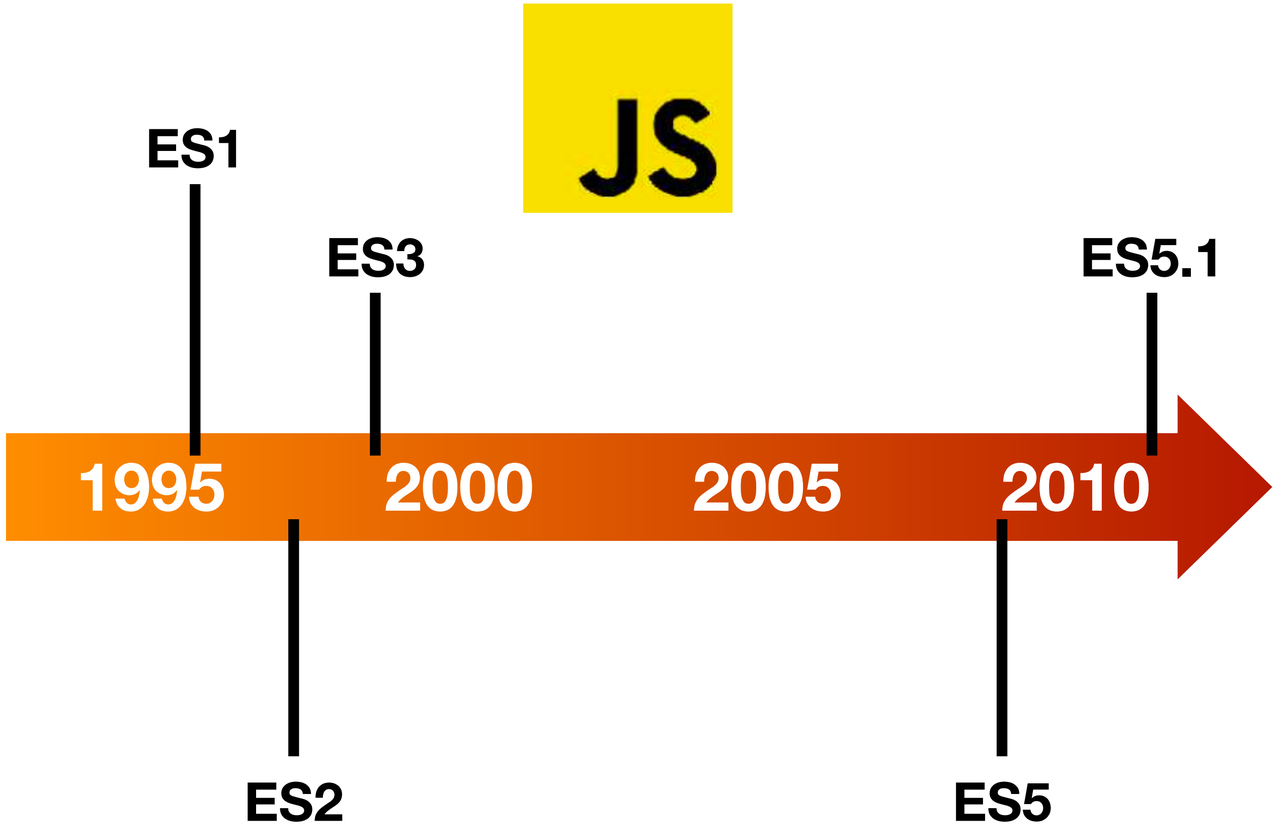
TC 39

Manual

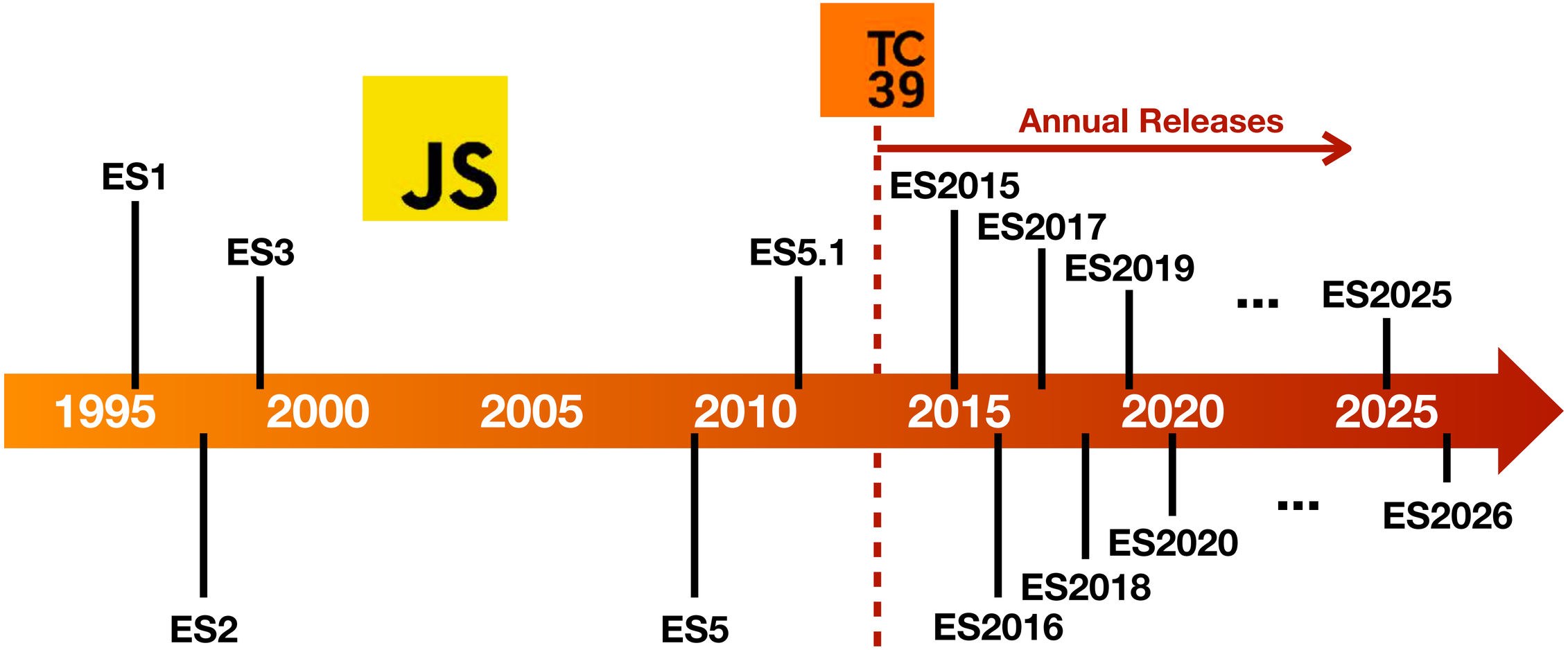
```
function isSupersetOf(other) {
  var O = aSet(this);
  var otherRec = getSetRecord(other);
  if (size(O) < otherRec.size) return false;
  var iterator = otherRec.getIterator();
  return iterateSimple(iterator,
    function (e) {
      if (!has(O, e)) {
        return iteratorClose(iterator,
                'normal', false);
      }
    }) !== false;
};
```

core-js

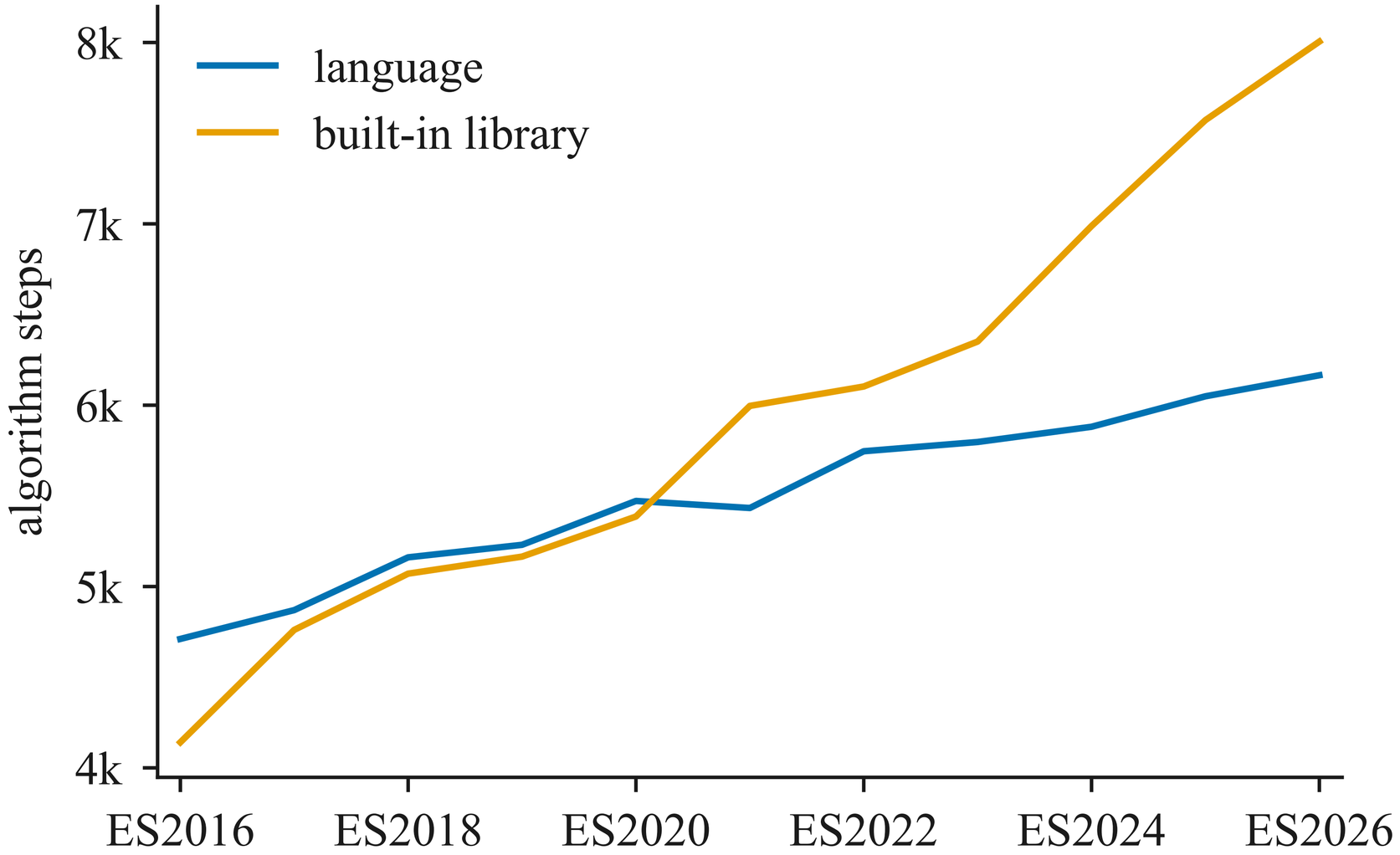
Problem - **Fast Evolving** JavaScript



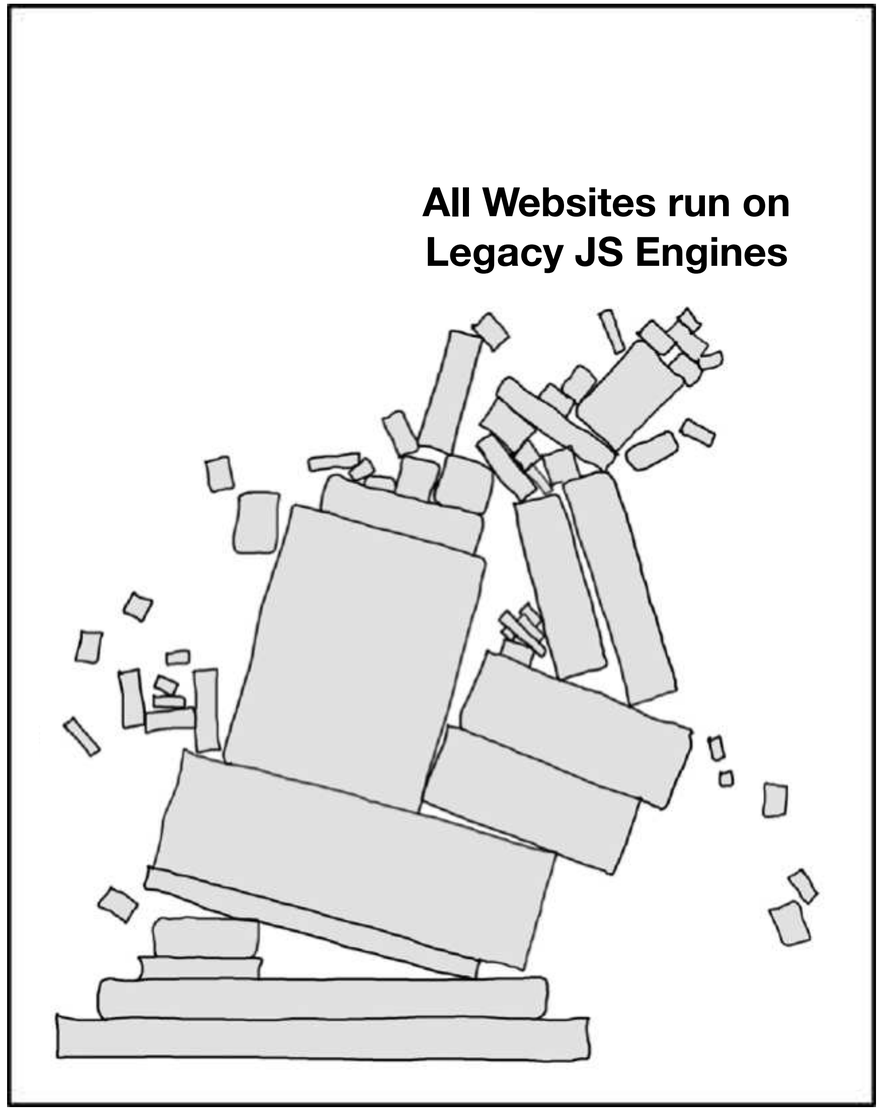
Problem - **Fast Evolving** JavaScript

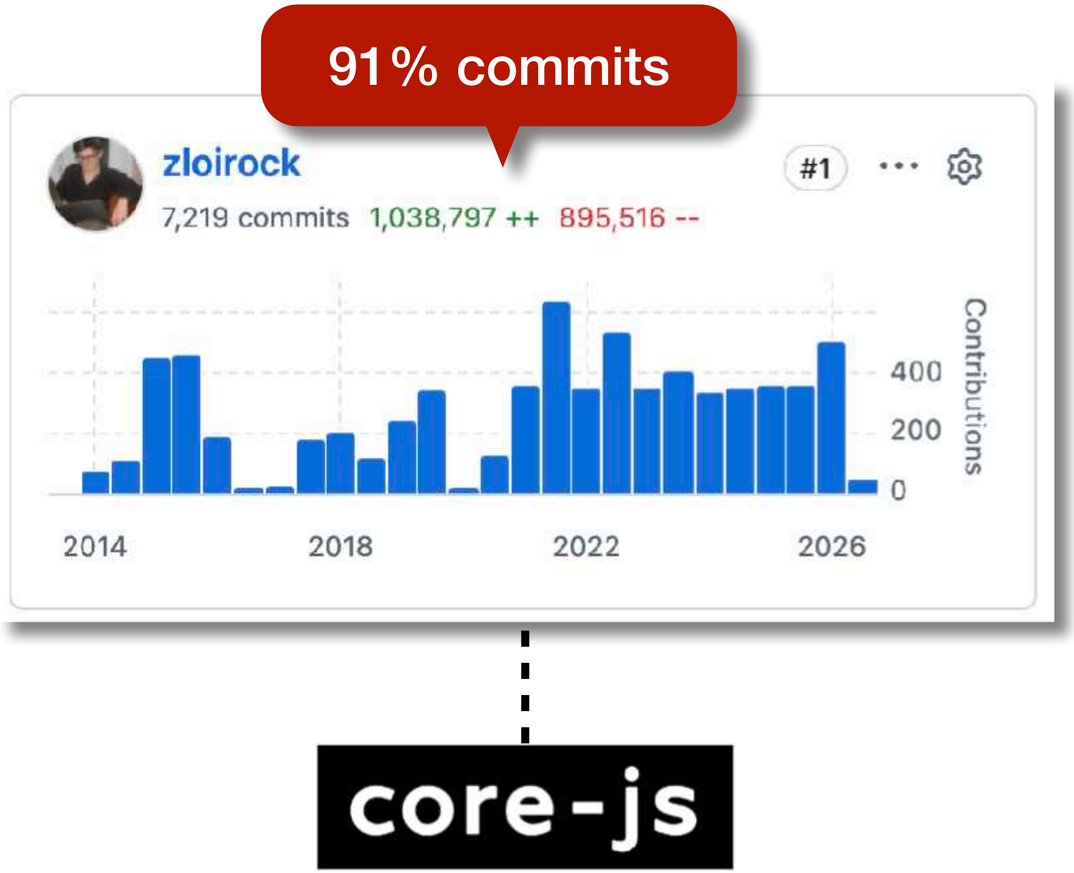


Problem - **Fast Evolving** JavaScript



Problem - Burden of **Manual** Implementation





Our Approach - **Automatic Extraction** from ECMA-262

Built-in

Set.prototype.isSupersetOf (*other*)
1. Let *O* be the **this** value.
2. Perform ? RequireInternalSlot(*O*, [[SetData]]).
3. Let *otherRec* be ? GetSetRecord(*other*).
4. If SetDataSize(*O*.[[SetData]]) < otherRec.[[Size]], return **false**.
5. Let *keysIter* be ? GetIteratorFromMethod(*otherRec*.[[SetObject]], *otherRec*.[[Keys]]).
6. Let *next* be NOT-STARTED.
7. Repeat, while *next* is not DONE,
   a. Set next to ? IteratorStepValue(*keysIter*).
   b. If *next* is not DONE, then
      i. If SetDataHas(*O*.[[SetData]], *next*) is **false**, then
         1. Perform ? IteratorClose(*keysIter*, NormalCompletion(UNUSED)).
         2. Return **false**.
8. Return **true**.

Extract

TC 39

Polyfill in **JavaScript**

Challenges - **Correctness** and **Efficiency**

Built-in

Set.prototype.isSupersetOf (*other*)
1. Let *O* be the **this** value.
2. Perform ? RequireInternalSlot(*O*, [[SetData]]).
3. Let *otherRec* be ? GetSetRecord(*other*).
4. If SetDataSize(*O*.[[SetData]]) < otherRec.[[Size]], return **false**.
5. Let *keysIter* be ? GetIteratorFromMethod(*otherRec*.[[SetObject]], *otherRec*.[[Keys]]).
6. Let *next* be NOT-STARTED.
7. Repeat, while *next* is not DONE,
   a. Set next to ? IteratorStepValue(*keysIter*).
   b. If *next* is not DONE, then
      i. If SetDataHas(*O*.[[SetData]], *next*) is **false**, then
         1. Perform ? IteratorClose(*keysIter*, NormalCompletion(UNUSED)).
         2. Return **false**.
8. Return **true**.

1) How to **correctly** handle *internal repr. of control flows*?

2) How to extract *algorithmically* **efficient** polyfill?

Extract

Polyfill in **JavaScript**

TC 39

Challenge 1 - **Correctness** (Internal Repr. of Control Flow)

A computation **X**

- is **normally** terminated with **a value** — **if X** is a **normal completion**
- **throws** an **exception** — **if X** is a **throw completion**

IteratorClose (*iteratorRecord*, *completion*)

...

3. Let *innerResult* be **GetMethod**(...).
4. If *innerResult* is a **normal completion**, then
   a. Let *return* be *innerResult*.[[Value]].
   b. If *return* is undefined, return ? *completion*.
   c. Set *innerResult* to **Call**(...).
5. If *completion* is a **throw completion**, return ? *completion*.
6. If *innerResult* is a **throw completion**, return ? *innerResult*.

...

8. Return ? *completion*.

TC 39

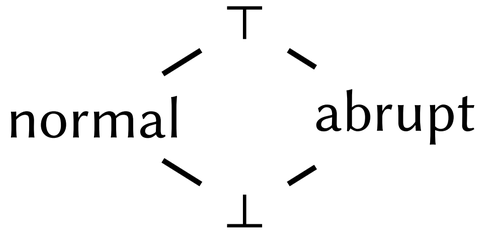
`return completion`
OR
`throw completion`

`throw completion`

`throw innerResult`

`return completion`

Solution 1 - **Completion Type Analysis**



IteratorClose (*iteratorRecord*, *completion*)
...
3. Let *innerResult* be **GetMethod**(...).

completion : ⊤
innerResult : ⊤

Completion

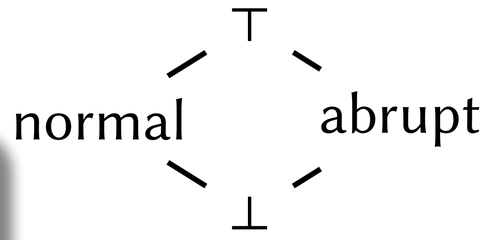
Completion

TC 39

```
function IteratorClose(iteratorRecord, completion, completion_flag) {
  ...
  try {
    var innerResult = GetMethod(...);
    var innerResult_flag = true;
  } catch(e) {
    var innerResult = e;
    var innerResult_flag = false;
  }
```

JS

Solution 1 - **Completion Type Analysis**



4. If *innerResult* is a **normal completion**, then
   a. Let *return* be *innerResult*.[[Value]].
   b. If *return* is undefined, return ? *completion*.
   c. Set *innerResult* to **Call**(...).

completion : ⊤
innerResult : normal

TC 39

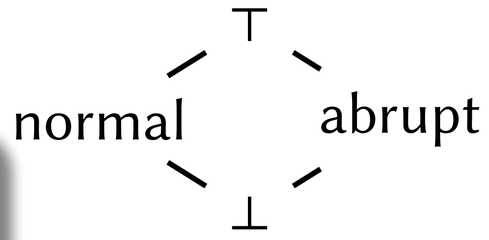
```
if (innerResult_flag) {
  ...



}
```

JS

Solution 1 - **Completion Type Analysis**



4. If *innerResult* is a **normal completion**, then
   a. Let *return* be *innerResult*.[[Value]].
   b. If *return* is undefined, return ? *completion*.
   c. Set *innerResult* to **Call**(...).

completion : ⊤
innerResult : normal

TC 39

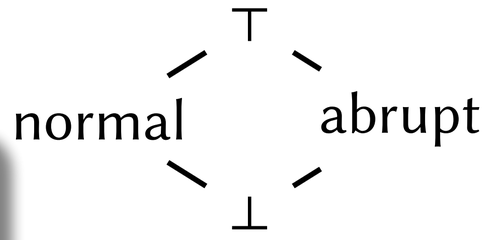
Completion

```
if (innerResult_flag) {
  ...
  if ( return === undefined) {
    if (completion_flag) return completion;
    else                 throw  completion;
  }

}
```

JS

Solution 1 - **Completion Type Analysis**



4. If *innerResult* is a **normal completion**, then
   a. Let *return* be *innerResult*.[[Value]].
   b. If *return* is undefined, return ? *completion*.
   c. Set *innerResult* to **Call**(...).

completion : ⊤
innerResult : normal

innerResult : ⊤

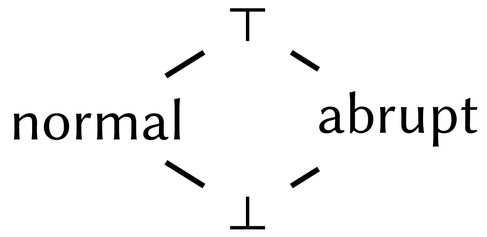
Completion

TC 39

```
if (innerResult_flag) {
  ...
  if ( return === undefined) {
    if (completion_flag) return completion;
    else                 throw  completion;
  }
  try { innerResult = Call(...); innerResult_flag = true; } catch ...
}
```

JS

Solution 1 - **Completion Type Analysis**



5. If *completion* is a **throw completion**, return ? *completion*.
6. If *innerResult* is a **throw completion**, return ? *innerResult*.
...
8. Return ? *completion*.

completion : normal

TC 39

completion : abrupt

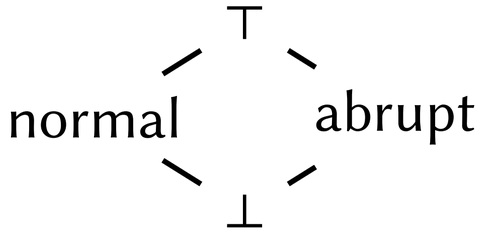
innerResult : abrupt

```
if (!completion_flag)  throw completion;

if (!innerResult_flag) throw innerResult;
```

JS

Solution 1 - **Completion Type Analysis**



5. If *completion* is a **throw completion**, return ? *completion*.
6. If *innerResult* is a **throw completion**, return ? *innerResult*.
...
8. Return ? *completion*.

completion : normal

TC 39

completion : abrupt

innerResult : abrupt

```
if (!completion_flag)  throw completion;

if (!innerResult_flag) throw innerResult;

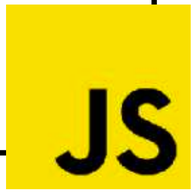
...

return completion;
```

JS

Solution 1 - **Completion Type Analysis**

```
function IteratorClose(iteratorRecord, completion, completion_flag) {
  ...
  try { var innerResult = GetMethod(...); var innerResult_flag = true; }
  catch(e) { var innerResult = e; var innerResult_flag = false; }
  if (innerResult_flag) {
    ...
    if (_return === undefined) {
      if (completion_flag) return completion;
      else                 throw  completion;
    }
    try { innerResult = Call(...); innerResult_flag = true; } catch ...
  }
  if (!completion_flag)  throw completion;
  if (!innerResult_flag) throw innerResult;
  ...
  return completion;
}
```



Challenge 2- **Efficiency**

$\mathcal{O}(n)$ **vs.** $\mathcal{O}(1)$

Set.prototype.add (*value*) **Linear** search

...

4. For each element *e* of *S*.[[SetData]], do

a. If *e* is not EMPTY and SameValue(*e*, *value*) is **true**, then

TC 39

Set.prototype. delete (*value*)

...

4. For each element *e* of *S*.[[SetData]], do

a. If *e* is not EMPTY and SameValue(*e*, *value*) is **true**, then

Set.prototype. has (*value*)

...

4. For each element *e* of *S*.[[SetData]], do

a. If *e* is not EMPTY and SameValue(*e*, *value*) is **true**, then

...

core-js

Set.prototype.add

Set.prototype.delete

Set.prototype.has

Solution 2 - **Rewrite Rules**

```
pattern: |
  1. For each element {{ elem : var }} of {{ base : ref }}, do
    1. If {{ elem }} is not EMPTY and SameValue({{ elem }},
          {{ value : var }}) is *true*, then
      1. {{ body : step }}
```

Set.prototype.add (*value*)

TC 39

...

4. For each element *e* of *S*.[[SetData]], do
   a. If *e* is not EMPTY and SameValue(*e*, *value*) is **true**, then
   ...

Solution 2 - **Rewrite Rules**

```
pattern: |
  1. For each element {{ elem : var }} of {{ base : ref }}, do
    1. If {{ elem }} is not EMPTY and SameValue({{ elem }},
          {{ value : var }}) is *true*, then
      1. {{ body : step }}
where:
  - isSetData(base)
  - { isSetData: " _.[[SetData]] " }
```

Set.prototype.add (*value*)

...

4. For each element *e* of S.[[SetData]], do
   a. If *e* is not EMPTY and SameValue(*e*, *value*) is **true**, then
   ...

TC 39

Solution 2 - **Rewrite Rules**

```
pattern: |
  1. For each element {{ elem : var }} of {{ base : ref }}, do
    1. If {{ elem }} is not ~empty~ and SameValue({{ elem }},
          {{ value : var }}) is *true*, then
      1. {{ body : step }}
where:
  - isSetData(base)
  - { isSetData: " _.[[SetData]] " }
replace: |
  1. If SET_DATA_HAS({{ base }}, {{ value }}) is *true*, then
    1. {{ body }}
```

Set.prototype.add (*value*)

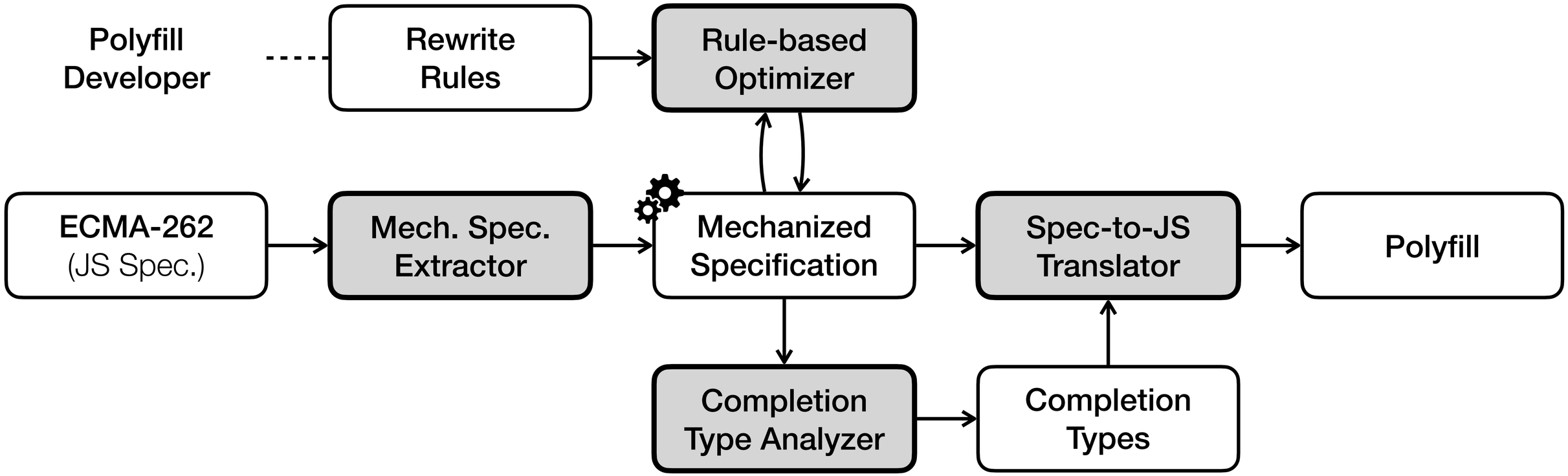
...

4. For each element *e* of *S*.[[SetData]], do
   a. If *e* is not EMPTY and SameValue(*e*, *value*) is **true**, then
   ...

TC 39

```
function Set.prototype.add(value) {
  ...
  if (SET_DATA_HAS(S.SetData, value)
      === true) {
    ...
  }
}
```

MiniPoly - Overall Structure



Evaluation

- Target **89 built-in APIs** in **ES2025**
- **Baselines** - `core-js` and `es-shims` (final release versions of 2025)
- Research Questions

  RQ1) Correctness

  RQ2) Reduced Manual Effort

  RQ3) Speed

  RQ4) Forward Compatibility

  RQ5) Real-World Usability

RQ1) Correctness

Inherently impossible test cases passed by **Polyfill** implementation

- Tested by **Test262** - official conformance tests
- **(# of Fail) core-js - 172 / es-shims - 1,058 / MiniPoly - 16**

			core-js		es-shims†		MiniPoly		Minipoly-opt	
Category	**# Built-ins**	**Tests**	**Pass (%)**	**Time**	**Pass (%)**	**Time**	**Pass (%)**	**Time**	**Pass (%)**	**Time**
Array	24	4,713	4,617 (98.0)	54.3s	4,365 (95.1)	81.5s	**4,713 (100.0)**	49.2s	**4,713 (100.0)**	51.5s
String	12	638	616 (96.6)	11.0s	612 (95.9)	12.4s	**638 (100.0)**	9.7s	**638 (100.0)**	9.5s
Set	18	748	710 (94.9)	16.1s	698 (93.8)	17.3s	**746 (99.7)**	13.8s	**746 (99.7)**	14.4s
Map	12	322	298 (92.5)	8.8s	258 (80.1)	8.4s	**320 (99.4)**	7.5s	**320 (99.4)**	7.7s
WeakMap	5	198	168 (84.8)	3.9s	–	–	**196 (99.0)**	3.6s	**196 (99.0)**	3.5s
WeakSet	5	166	142 (85.5)	3.9s	–	–	**164 (98.8)**	3.8s	**164 (98.8)**	3.1s
Promise	13	1,264	1,154 (91.3)	21.6s	0 (0.0)	10.9s	**1,256 (99.4)**	18.5s	**1,256 (99.4)**	18.9s
Total	**89**	**8,049**	**7,705 (95.7)**	**119.5s**	**5,933 (87.4)**	**130.5s**	**8,033 (99.8)**	**106.1s**	**8,033 (99.8)**	**108.7s**

RQ2) Reduced Manual Effort

		Baseline		MiniPoly		MiniPoly-opt		
Object	**#**	**core-js**	**es-shims**	**Auto**	**Man.**	**Auto**	**Man.**	**Rules**
Array	24	2,298	8,978	1,332	317			
String	12	1,281	6,885	418	130			
Set	18	2,313	5,880	864	190	861	283	114 (11)
WeakSet	5	1,882	–	387	123	379	214	
Map	12	1,986	3,062	756	217	747	327	81 (6)
WeakMap	5	2,034	–	446	115	436	231	
Promise	13	2,518	8,031	1,511	203			
Total†	**89**	**14,312**	**32,836**	**5,714**	**1,295**	**5,684**	**1,705**	**195 (17)**

3.94%

9.04%

RQ2) Reduced Manual Effort

		Baseline		MiniPoly		MiniPoly-opt		
Object	**#**	**core-js**	**es-shims**	**Auto**	**Man.**	**Auto**	**Man.**	**Rules**
Array	24	2,298	8,978	1,332	317			
String	12	1,281	6,885	418	130			
Set	18	2,313	5,880	864	190	861	283	114 (11)
WeakSet	5	1,882	–	387	123	379	214	
Map	12	1,986	3,062	756	217	747	327	81 (6)
WeakMap	5	2,034	–	446	115	436	231	
Promise	13	2,518	8,031	1,511	203			
Total†	**89**	**14,312**	**32,836**	**5,714**	**1,295**	**5,684**	**1,705**	**195 (17)**

5.79%

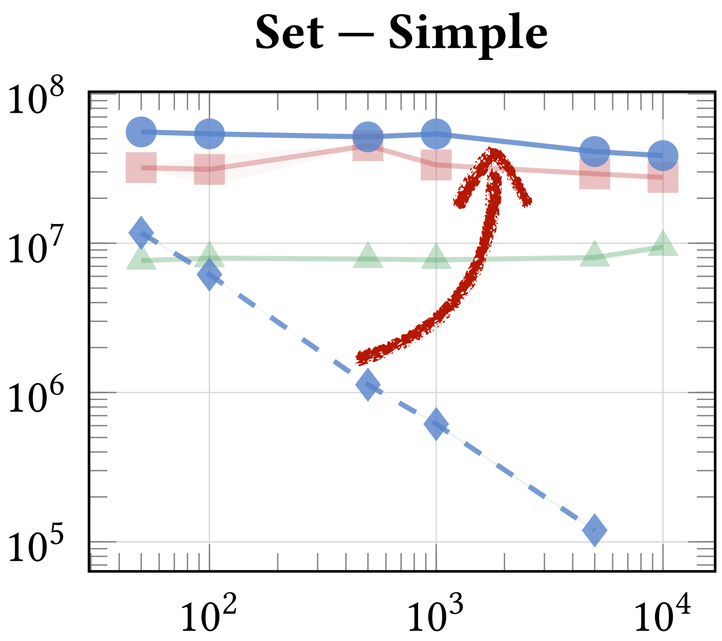
13.28%

17 rules

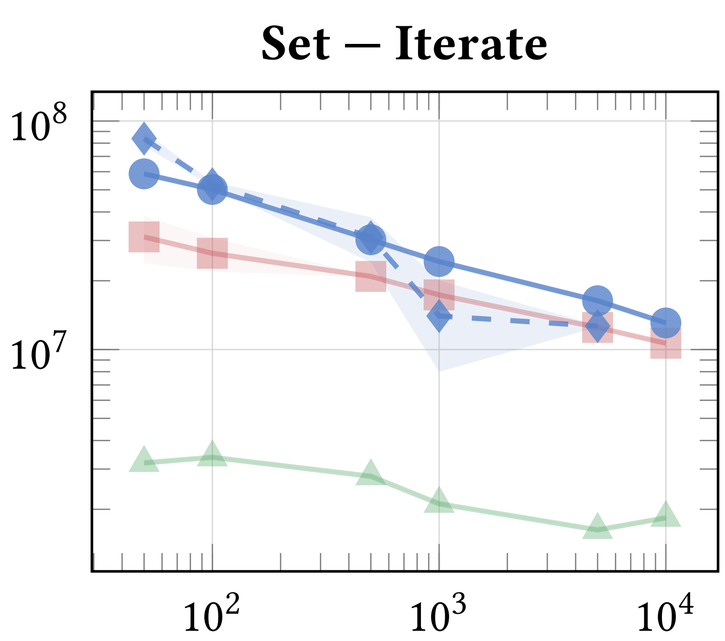
RQ3) Speed

- We measured **throughput** (*y-axis* = ops/sec) across **sizes** (*x-axis* = N)

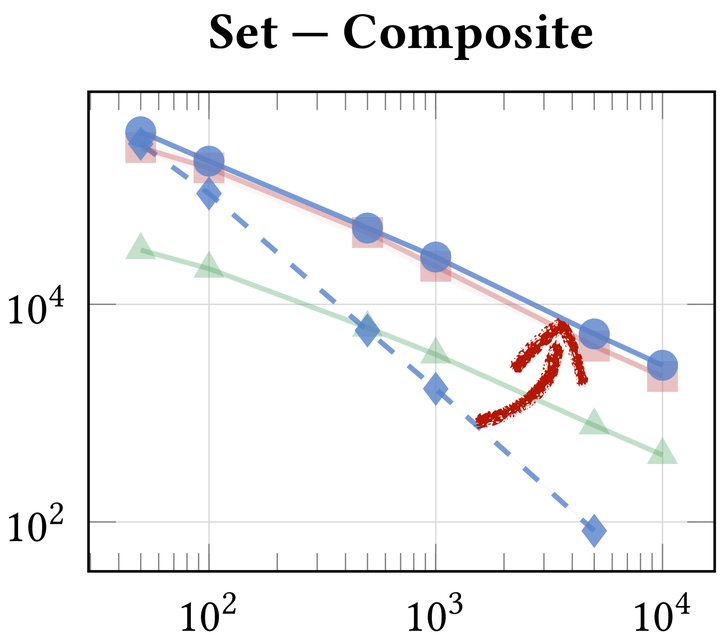
O(N) → O(1)



O(N) → O(N)



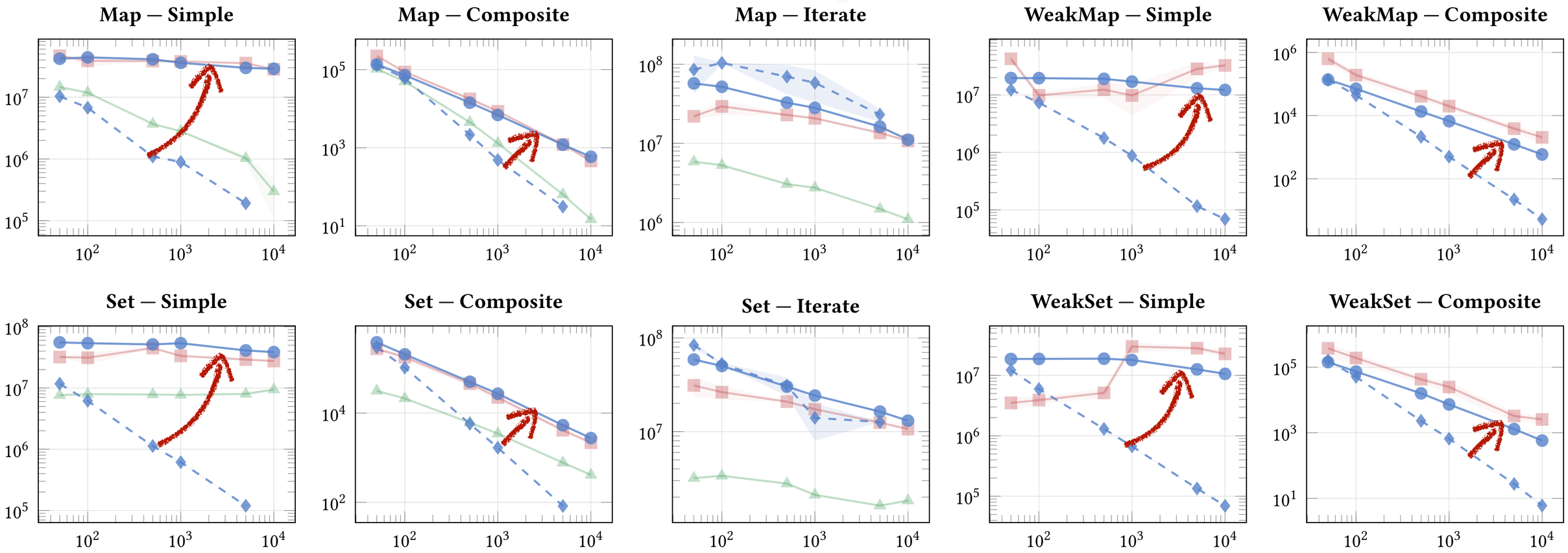
O(N²) → O(N)





RQ3) Speed

Competitive with baselines in most cases

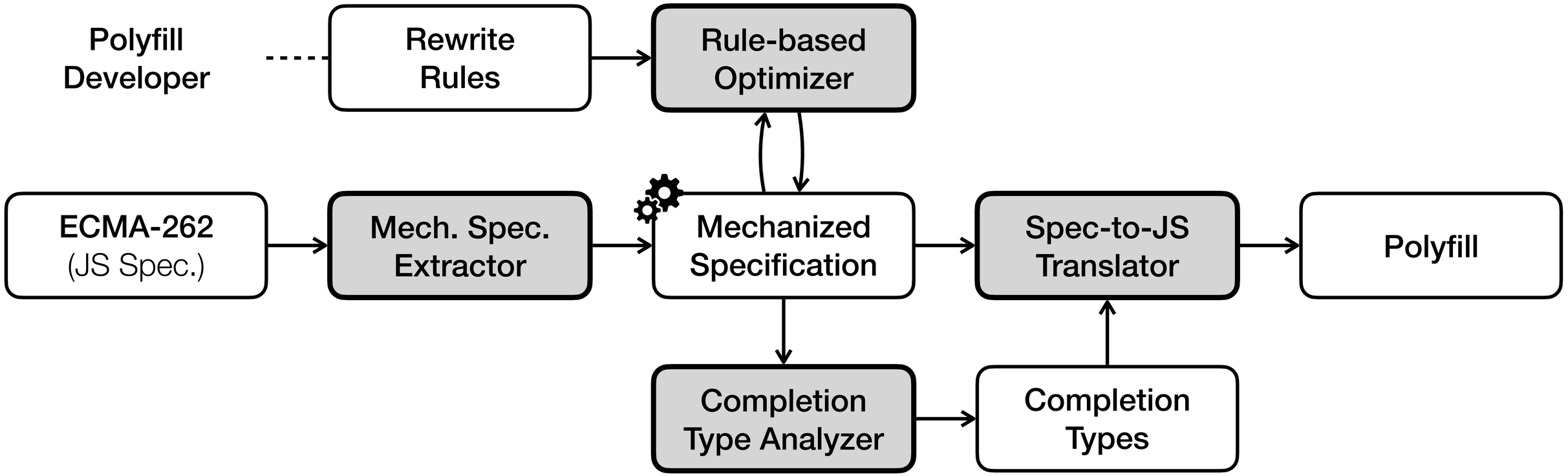


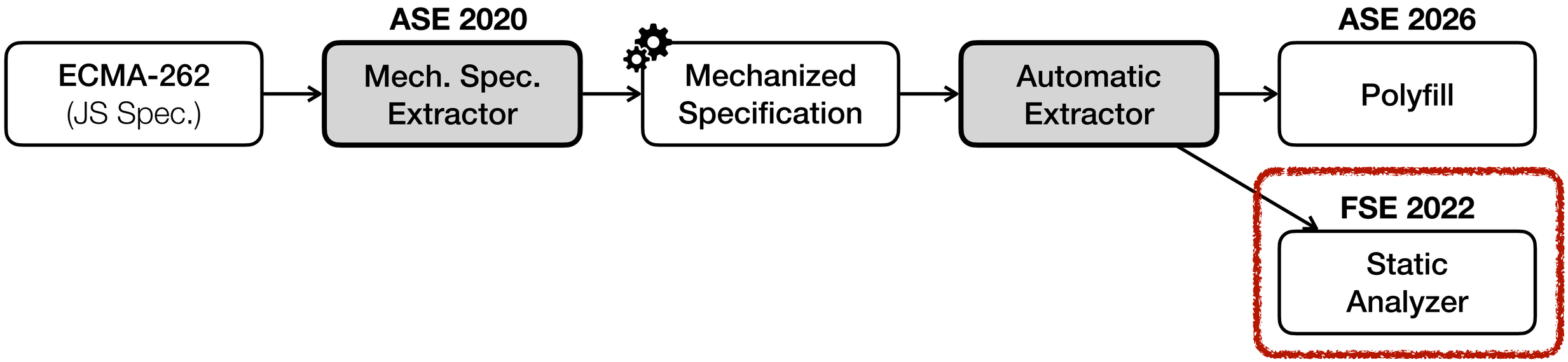
RQ4) Forward Compatibility

- Supports Built-in APIs in **ES2026** and **Stage 2.7** with **100% correctness** and **compatible speed** with manual baselines

			Correctness (Pass %)				**Throughput (ops/sec)** — N = 5,000			
Proposal	**Method**	**Tests**	core-js	es-shims	MiniPoly	MiniPoly-opt	core-js	es-shims	MiniPoly	MiniPoly-opt
Upsert (ES2026)	Map.prototype									
	.getOrInsert	28	26 (92.9)	24 (85.7)	**28 (100.0)**	**28 (100.0)**	13.7M / 2.0M	61.2K	107K	8.2M / 2.5M
	.getOrInsertComputed	37	35 (94.6)	33 (89.2)	**37 (100.0)**	**37 (100.0)**	11.7M / 1.8M	40.8K	77.7K	6.5M / 1.9M
	WeakMap.prototype									
	.getOrInsert	34	26 (76.5)	—	**34 (100.0)**	**34 (100.0)**	18.4M / 3.1M	—	104K	8.2M / 2.3M
	.getOrInsertComputed	43	32 (74.4)	—	**43 (100.0)**	**43 (100.0)**	5.7M / 1.2M	—	73.1K	5.0M / 1.8M
Await Dictionary (Stage 2.7)	Promise									
	.allKeyed	12	—	—	**12 (100.0)**	**12 (100.0)**	—	—	—	—
	.allSettledKeyed	12	—	—	**12 (100.0)**	**12 (100.0)**	—	—	—	—
Total		**166**	**119 (83.8)**	**57 (87.7)**	**166 (100.0)**	**166 (100.0)**	11.3M / 1.9M	50.0K	89.3K	6.8M / 2.1M

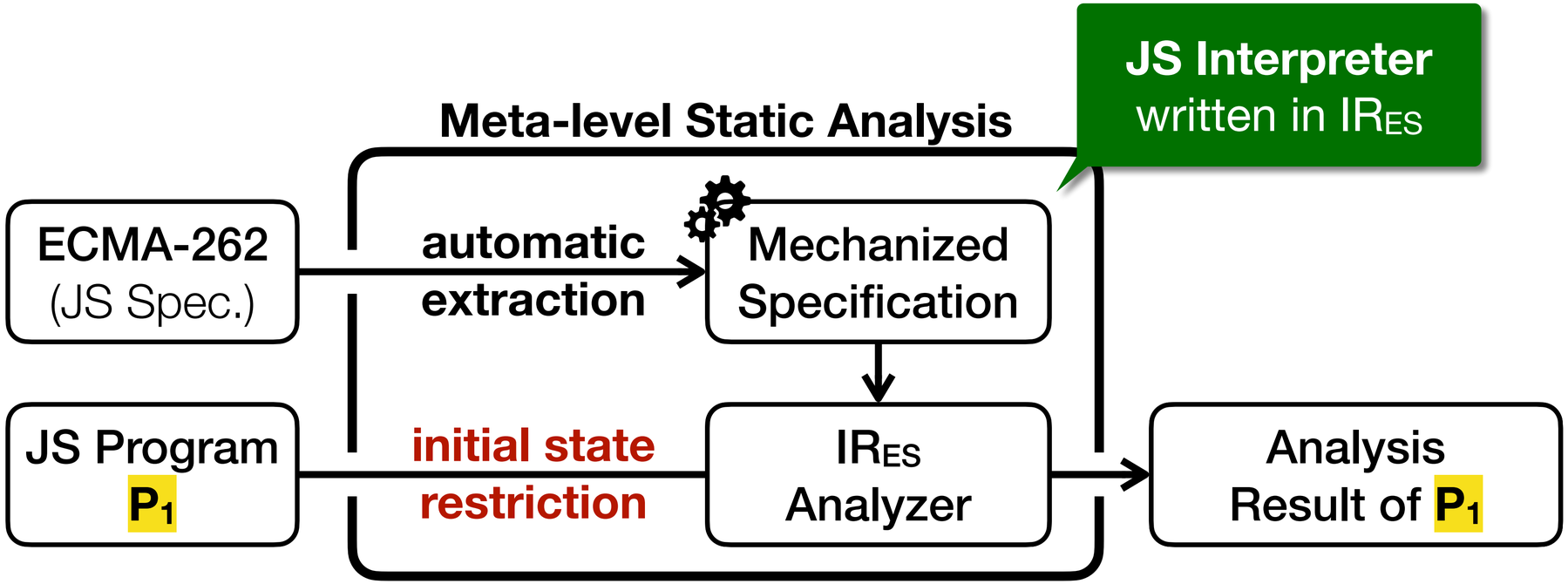
-23 -8


Polyfill
Developer
Rewrite
Rules
Rule-based
Optimizer
ECMA-262
(JS Spec.)
Mech. Spec.
Extractor
Mechanized
Specification
Spec-to-JS
Translator
Polyfill
Completion
Type Analyzer
Completion
Types


ASE 2020
ASE 2026
ECMA-262
(JS Spec.)
Mech. Spec.
Extractor
Mechanized
Specification
Automatic
Extractor
Polyfill
FSE 2022
Static
Analyzer

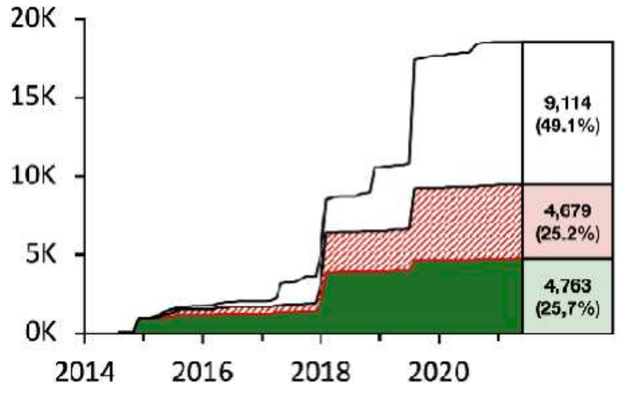
Meta-Level Static Analysis

How to perform **static analysis** on **JavaScript programs** using **language specification**?

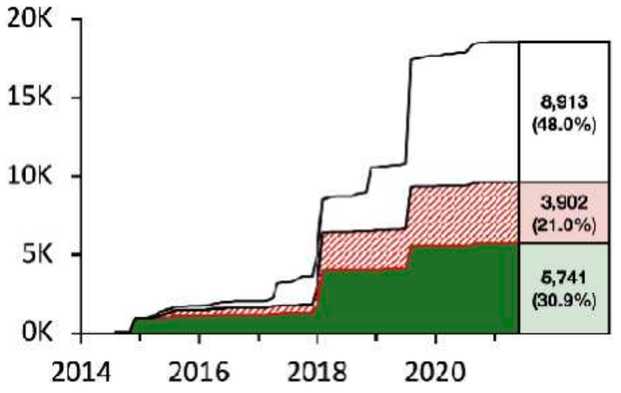


Evaluation

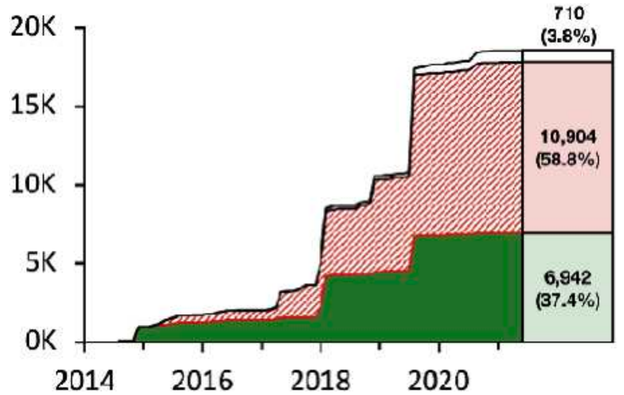
Manual



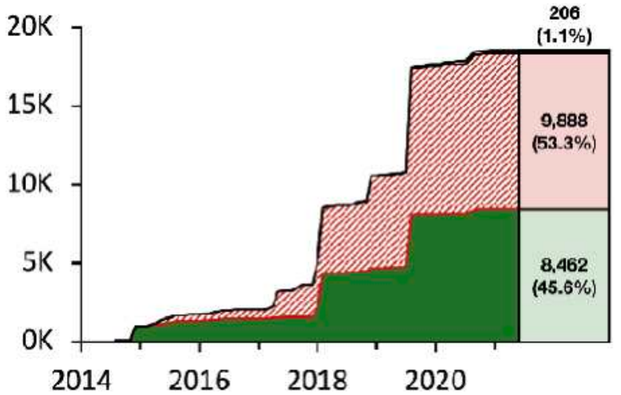
TAJS



SAFE

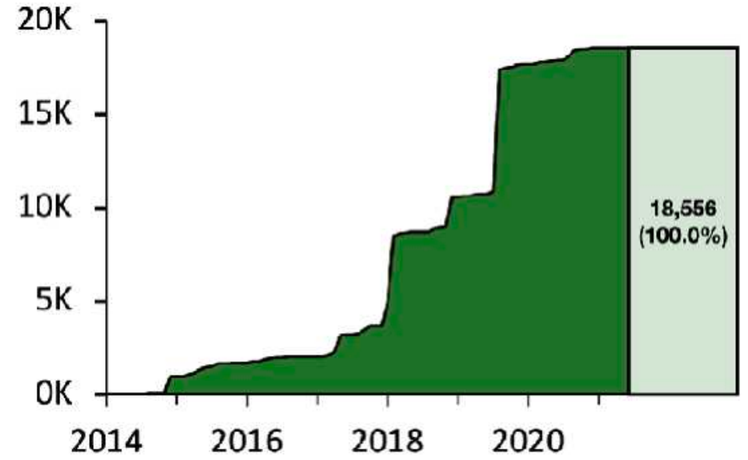


TAJS with Babel

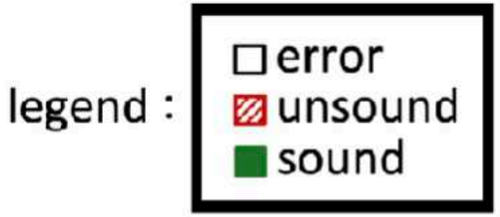


SAFE with Babel

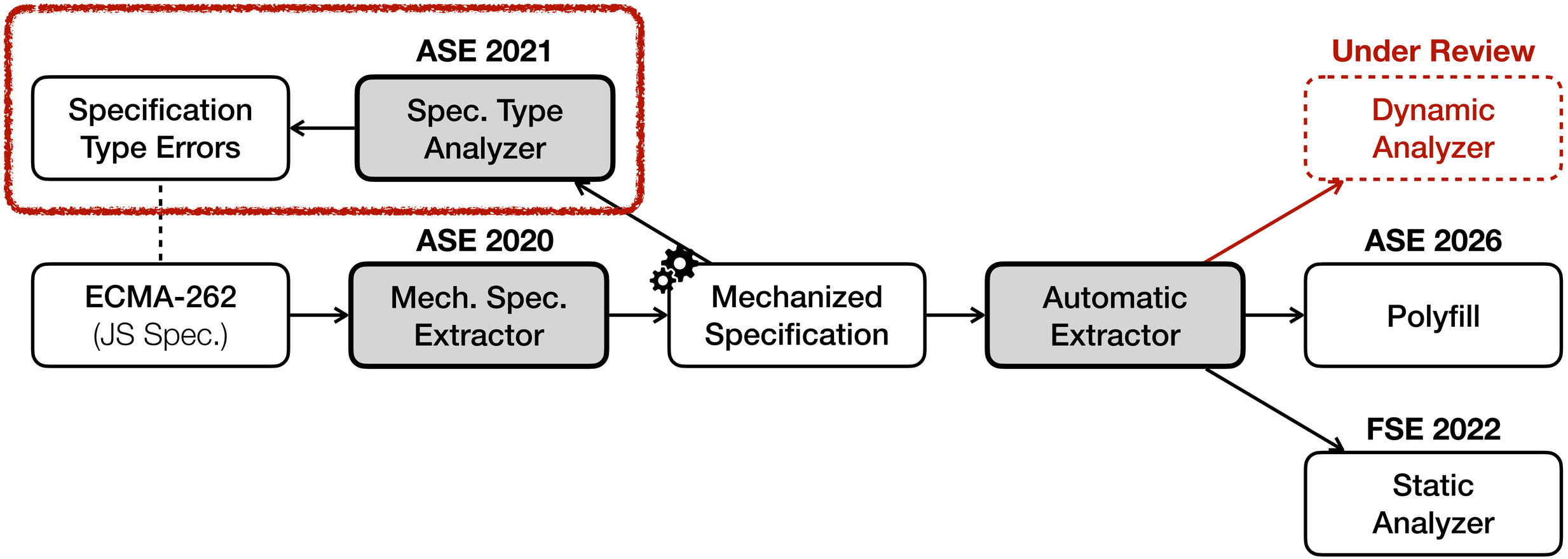
Derived



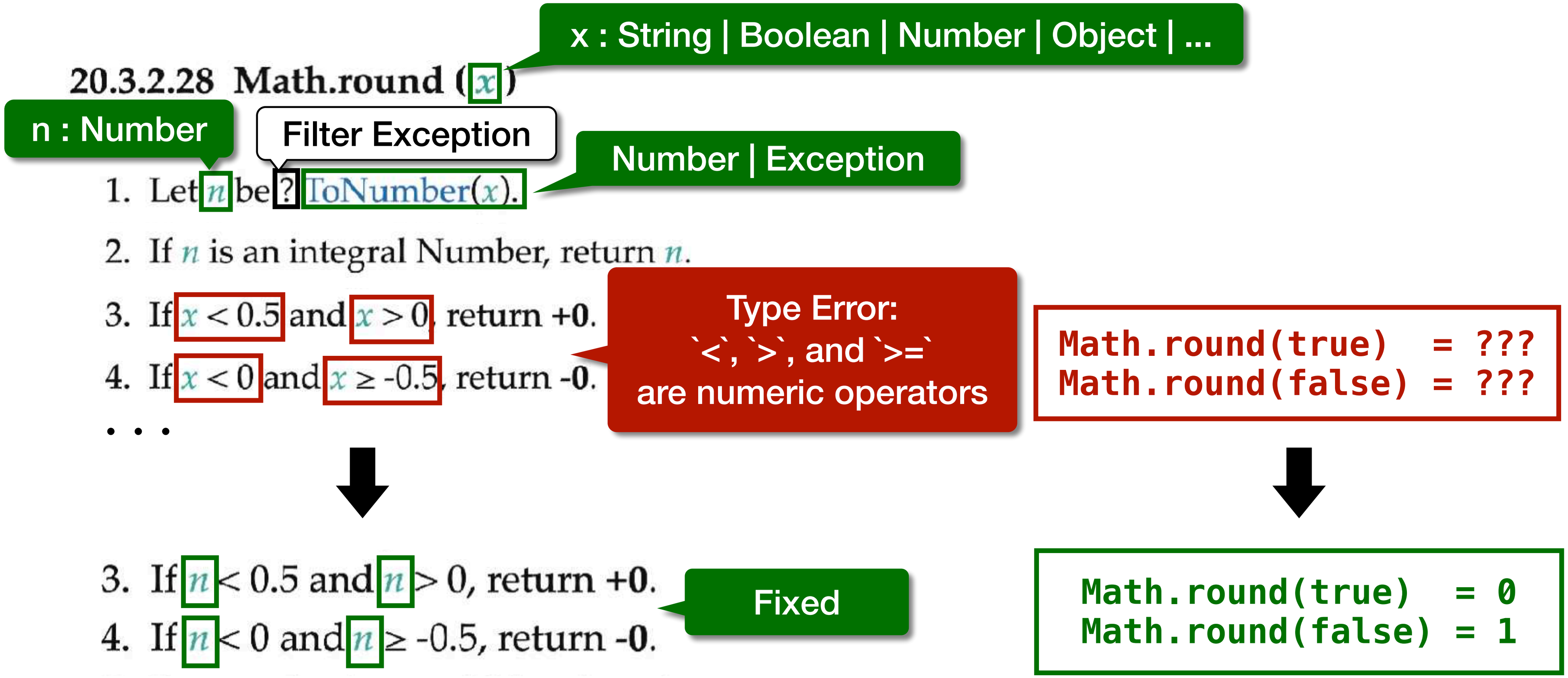
Derived Analyzer



x-axis : creation time (year)
y-axis : # tests


ASE 2021
Specification Type Errors
Spec. Type Analyzer
Under Review
Dynamic Analyzer
ASE 2020
ASE 2026
ECMA-262 (JS Spec.)
Mech. Spec. Extractor
Mechanized Specification
Automatic Extractor
Polyfill
FSE 2022
Static Analyzer

Specification Type Analysis



https://github.com/tc39/ecma262/tree/575149cfd77aebcf3a129e165bd89e14caafc31c

Evaluation

- Type analysis on **864 versions** of ECMA-262 in **3 years**

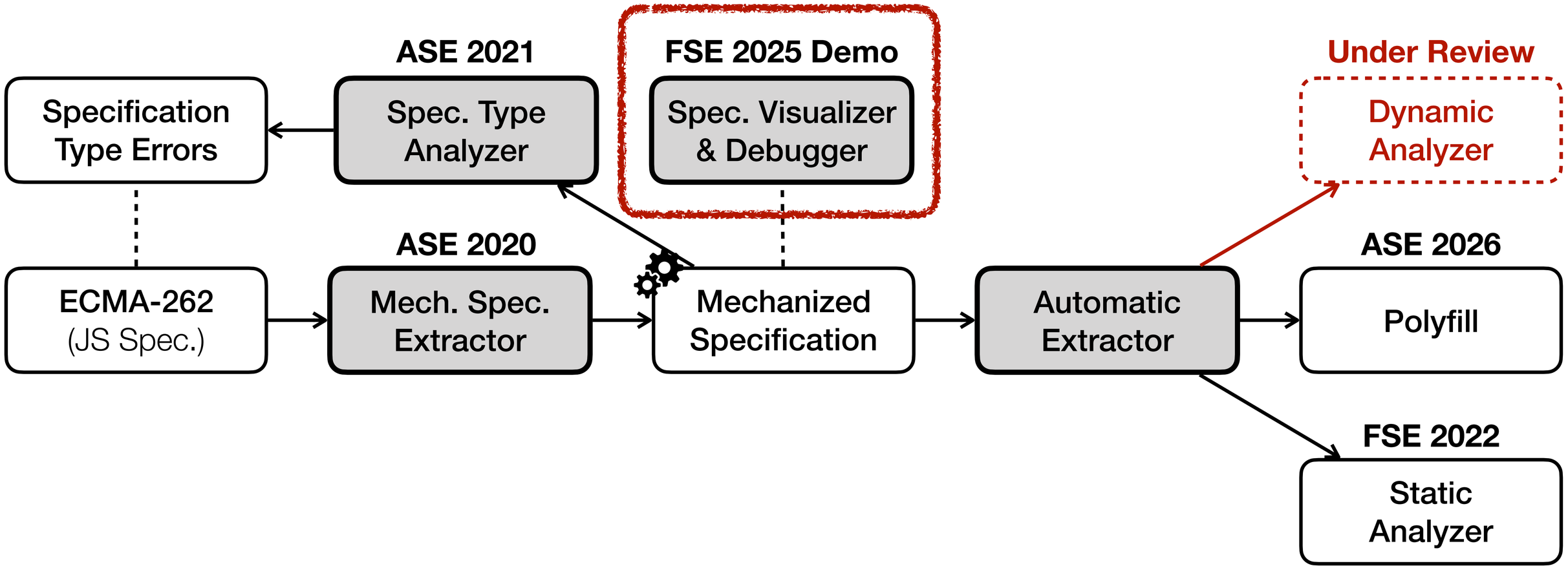
Checker	Bug Kind	Precision = (# True Bugs) / (# Detected Bugs)					
		no-refine		refine		Δ	
Reference	UnknownVar	62 / 106	17 / 60	63 / 78	17 / 31	+1 / -28	/ -29
	DuplicatedVar		45 / 46		46 / 47		+1 / +1
Arity	MissingParam			4 / 4	4 /		/
Assertion	Assertion			4 / 31	4 / 3		/ -25
Operand	NoNumber			22 / 44	2 /		/ -59
	Abrupt		20 / 48		20 / 38		/ -10
Total		92 / 279 (33.0%)		93 / 157 (59.2%)		+1 / -122 (+26.3%)	

93 Errors Detected

59.2% Precision

Name	Feature	#	Checker	Created	Life Span
ES12-1	Switch	3	Reference	2015-09-22	1,996 days
ES12-2	Try	3	Reference	2015-09-22	1,996 days
ES12-3	Arguments	1	Reference	2015-09-22	1,996 days
ES12-4	Array	2	Reference	2015-09-22	1,996 days
ES12-5	Async	1	Reference	2015-09-22	1,996 days
ES12-6	Class	1	Reference	2015-09-22	1,996 days
ES12-7	Branch	1	Reference	2015-09-22	1,996 days
ES12-8	Arguments	2	Operand	2015-12-16	1,910 days

14 New Bugs In ES2021


ASE 2021
FSE 2025 Demo
Under Review
Specification Type Errors
Spec. Type Analyzer
Spec. Visualizer & Debugger
Dynamic Analyzer
ASE 2020
ASE 2026
ECMA-262 (JS Spec.)
Mech. Spec. Extractor
Mechanized Specification
Automatic Extractor
Polyfill
FSE 2022
Static Analyzer

ESMeta / Double Debugger Playground

Docs GitHub READY ES2026

RUN QUIT SPEC STEP OVER OUT CONTINUE SPEC BACK BACK OVER BACK OUT REWIND JS STEP

AST STEP OVER OUT

JavaScript Editor

```
1 const a = new Set
  ([1, 2, 3]);
2 const b = new Set
  ([1, 2]);
3 a.isSupersetOf(b);
```

ECMAScript Specification

Context Not Found

State Env Heap Breaks Callstack

Disabled. Start debugger to use.

JavaScript Editor

```
1 const a = new Set
  ([1, 2, 3]);
2 const b = new Set
  ([1, 2]);
3 a.isSupersetOf(b);
```

ECMAScript Specification

Set.prototype.isSupersetOf(this, ArgumentsList, NewTarget)

1. Let *O* be the **this** value.
2. Perform ? RequireInternalSlot(*O* , [[SetData]]).
3. Let *otherRec* be ? GetSetRecord(*other*).
4. If SetDataSize(*O* .[[SetData]]) < *otherRec* .[[Size]], return **false**.
5. Let *keysIter* be ? GetIteratorFromMethod(*otherRec* .[[SetObject]], *otherRec* .[[Keys]]).
6. Let *next* be not-started.
7. Repeat, while *next* is not done,
   a. Set *next* to ? IteratorStepValue(*keysIter*).
   b. If *next* is not done, then
      i. If SetDataHas(*O* .[[SetData]], *next*) is **false**, then
         i. Perform ? IteratorClose(*keysIter* ,

State Env Heap Breaks Callstack

Breakpoints USING BREAKPOINTS

search by name

Step	Name	Enable	Remove

No breakpoints. Add Breakpoint by clicking on steps in spec viewer or by searching name

RUN | QUIT | SPEC STEP | OVER | OUT | CONTINUE | SPEC BACK | BACK OVER | BACK OUT | REWIND | JS STEP

AST STEP | OVER | OUT

JavaScript Editor

```
1 const a = new Set
  ([1, 2, 3]);
2 const b = new Set
  ([1, 2]);
3 a.isSupersetOf(b);
```

ECMAScript Specification

Set.prototype.isSupersetOf(this, ArgumentsList, NewTarget)

1. Let *O* be the **this** value.
2. Perform ? RequireInternalSlot(*O* , [[SetData]]).
3. Let *otherRec* be ? GetSetRecord(*other*).
4. If SetDataSize(*O* .[[SetData]]) < *otherRec* .[[Size]], return **false**.
5. Let *keysIter* be ? GetIteratorFromMethod(*otherRec* .[[SetObject]], *otherRec* .[[Keys]]).
6. Let *next* be not-started.
7. Repeat, while *next* is not done,
   a. Set *next* to ? IteratorStepValue(*keysIter*).
   b. If *next* is not done, then
      i. If SetDataHas(*O* .[[SetData]], *next*) is **false**, then
         i. Perform ? IteratorClose(*keysIter* ,

State | Env | Heap | Breaks | Callstack

Specification Environment

- **ArgumentsList** : List
- **NewTarget** : undefined
- **0** : Record[Object]
- **other** : Record[Object]
- **this** : Record[Object]

JavaScript Environment

No environment variables.

RUN | QUIT | SPEC STEP | OVER | OUT | CONTINUE | SPEC BACK | BACK OVER | BACK OUT | REWIND | JS STEP

AST STEP | OVER | OUT

JavaScript Editor

```
1 const a = new Set
  ([1, 2, 3]);
2 const b = new Set
  ([1, 2]);
3 a.isSupersetOf(b);
```

ECMAScript Specification

Set.prototype.isSupersetOf(this, ArgumentsList, NewTarget)

1. Let *O* be the **this** value.
2. Perform ? RequireInternalSlot(*O* , [[SetData]]).
3. Let *otherRec* be ? GetSetRecord(*other*).
4. If SetDataSize(*O* .[[SetData]]) < *otherRec* .[[Size]], return **false**.
5. Let *keysIter* be ? GetIteratorFromMethod(*otherRec* .[[SetObject]], *otherRec* .[[Keys]]).
6. Let *next* be not-started.
7. Repeat, while *next* is not done,
   a. Set *next* to ? IteratorStepValue(*keysIter*).
   b. If *next* is not done, then
      i. If SetDataHas(*O* .[[SetData]], *next*) is **false**, then
         i. Perform ? IteratorClose(*keysIter* ,

State | Env | Heap | Breaks | Callstack

Specification Environment

- **ArgumentsList** : List
- **NewTarget** : undefined
- **0** : Record[Object]
- **other** : Record[Object]
- **this** : Record[Object]

JavaScript Environment

No environment variables.

RUN | QUIT | SPEC STEP | OVER | OUT | CONTINUE | SPEC BACK | BACK OVER | BACK OUT | REWIND | JS STEP

AST STEP | OVER | OUT

JavaScript Editor

```
1 const a = new Set
  ([1, 2, 3]);
2 const b = new Set
  ([1, 2]);
3 a.isSupersetOf(b);
```

ECMAScript Specification

Set.prototype.isSupersetOf(this, ArgumentsList, NewTarget)

1. Let *O* be the **this** value.
2. Perform ? RequireInternalSlot(*O* , [[SetData]]).
3. Let *otherRec* be ? GetSetRecord(*other*).
4. If SetDataSize(*O* .[[SetData]]) < *otherRec* .[[Size]], return **false**.
5. Let *keysIter* be ? GetIteratorFromMethod(*otherRec* .[[SetObject]], *otherRec* .[[Keys]]).
6. Let *next* be not-started.
7. Repeat, while *next* is not done,
   a. Set *next* to ? IteratorStepValue(*keysIter*).
   b. If *next* is not done, then
      i. If SetDataHas(*O* .[[SetData]], *next*) is **false**, then
         i. Perform ? IteratorClose(*keysIter* ,

State | Env | Heap | Breaks | Callstack

Specification Environment

- **ArgumentsList** : List
- **NewTarget** : undefined
- **O** : Record[Object]
- **other** : Record[Object]
- **otherRec** : Record[SetRecord]
- **this** : Record[Object]

JavaScript Environment

No environment variables.

JavaScript Editor

```
1 const a = new Set
  ([1, 2, 3]);
2 const b = new Set
  ([1, 2]);
3 a.isSupersetOf(b);
```

ECMAScript Specification

Set.prototype.isSupersetOf(this, ArgumentsList, NewTarget)

1. Let *O* be the **this** value.
2. Perform ? RequireInternalSlot(*O* , [[SetData]]).
3. Let *otherRec* be ? GetSetRecord(*other*).
4. If SetDataSize(*O* .[[SetData]]) < *otherRec* .[[Size]], return **false**.
5. Let *keysIter* be ? GetIteratorFromMethod(*otherRec* .[[SetObject]], *otherRec* .[[Keys]]).
6. Let *next* be not-started.
7. Repeat, while *next* is not done,
   a. Set *next* to ? IteratorStepValue(*keysIter*).
   b. If *next* is not done, then
      i. If SetDataHas(*O* .[[SetData]], *next*) is **false**, then
         i. Perform ? IteratorClose(*keysIter* ,

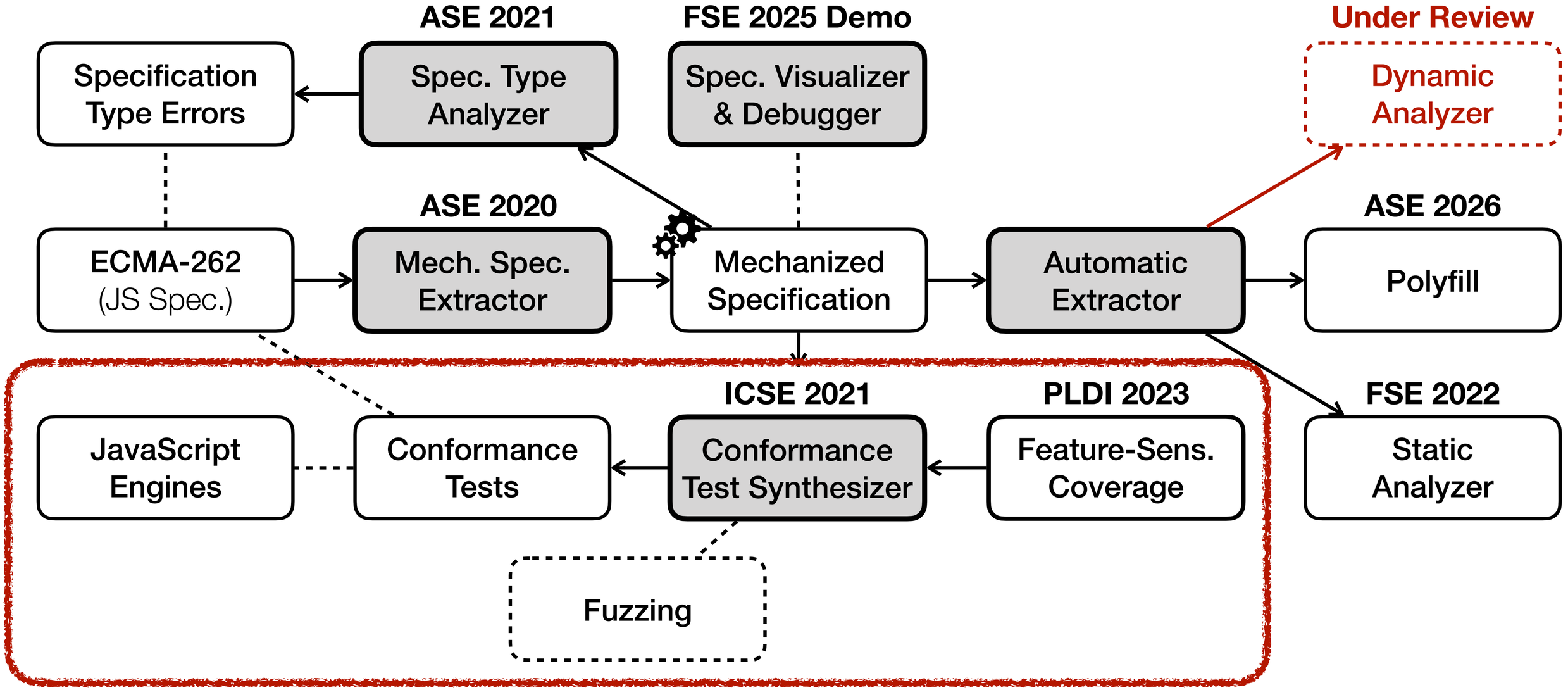
State Env Heap Breaks Callstack

Specification Environment

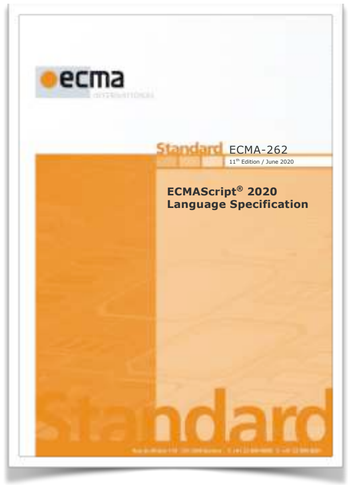
- **ArgumentsList** : List
- **NewTarget** : undefined
- **O** : Record[Object]
- **other** : Record[Object]
- **otherRec** : Record[SetRecord]
  - **Size** : 2
  - **Keys** : Record[BuiltinFunctionObject]
  - **Has** : Record[BuiltinFunctionObject]
  - **SetObject** : Record[Object]
- **this** : Record[Object]

JavaScript Environment

No environment variables.

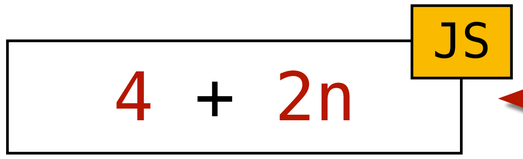

ASE 2021
FSE 2025 Demo
Under Review
Specification Type Errors
Spec. Type Analyzer
Spec. Visualizer & Debugger
Dynamic Analyzer
ASE 2020
ASE 2026
ECMA-262 (JS Spec.)
Mech. Spec. Extractor
Mechanized Specification
Automatic Extractor
Polyfill
ICSE 2021
PLDI 2023
FSE 2022
JavaScript Engines
Conformance Tests
Conformance Test Synthesizer
Feature-Sens. Coverage
Static Analyzer
Fuzzing

Conformance of JavaScript Engines



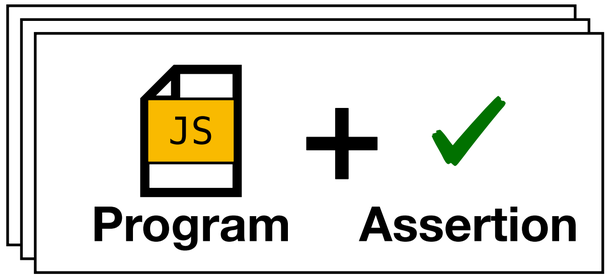
ECMA-262
(JavaScript Spec.)

Example:



Assertion

`TypeError`



Conformance Tests

Conformance

Test262
(Official Test Suite)



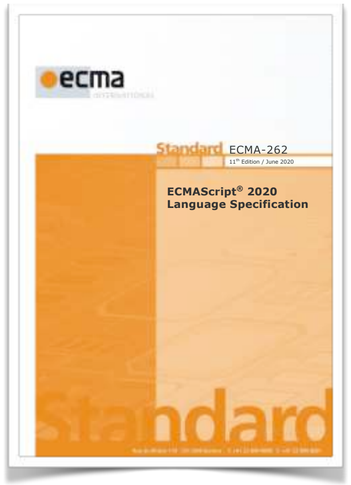
GraalVM

`QuickJS`

moddable

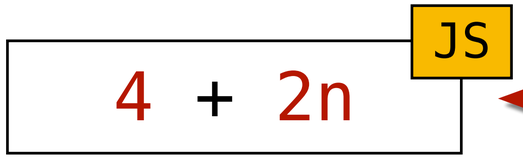
JavaScript Engines

Problem - **Manual Approach**



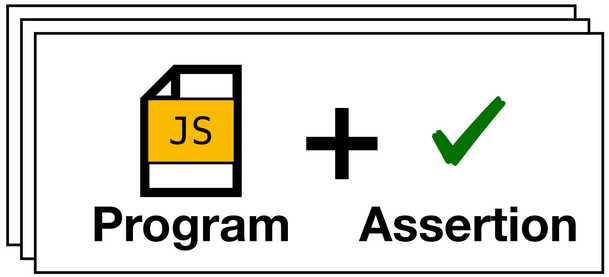
ECMA-262
(JavaScript Spec.)

Example:



Assertion

`TypeError`



Conformance Tests

Conformance

Test262
(Official Test Suite)

`Manual`



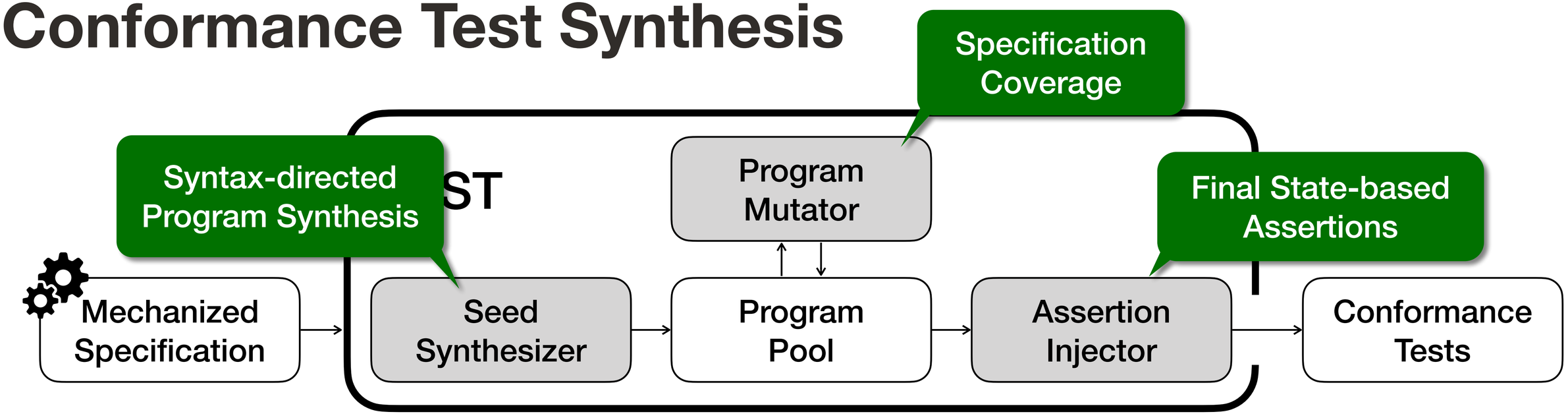
GraalVM

`QuickJS`

moddable

JavaScript Engines

Conformance Test Synthesis



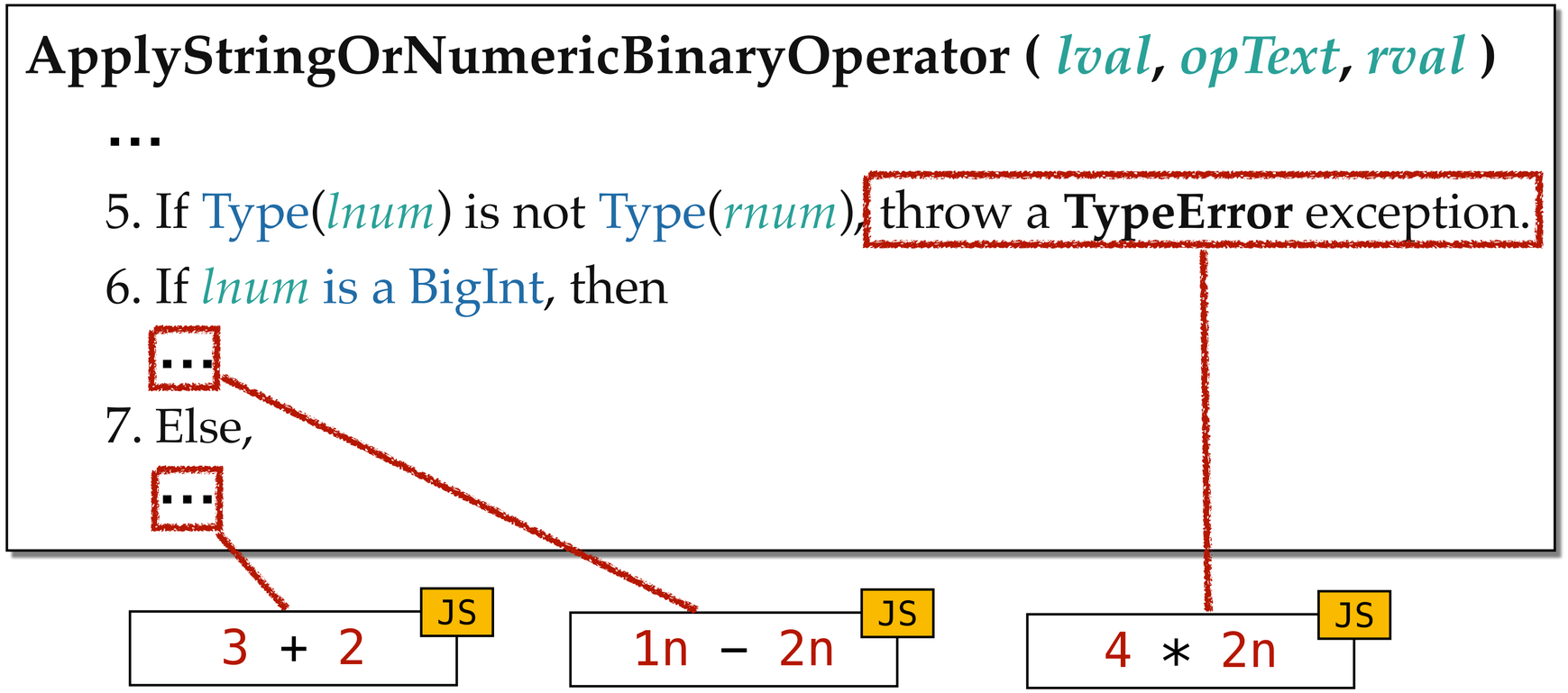
Program Pool

```
let x = 1 + 2;
assert(x == 3);
```

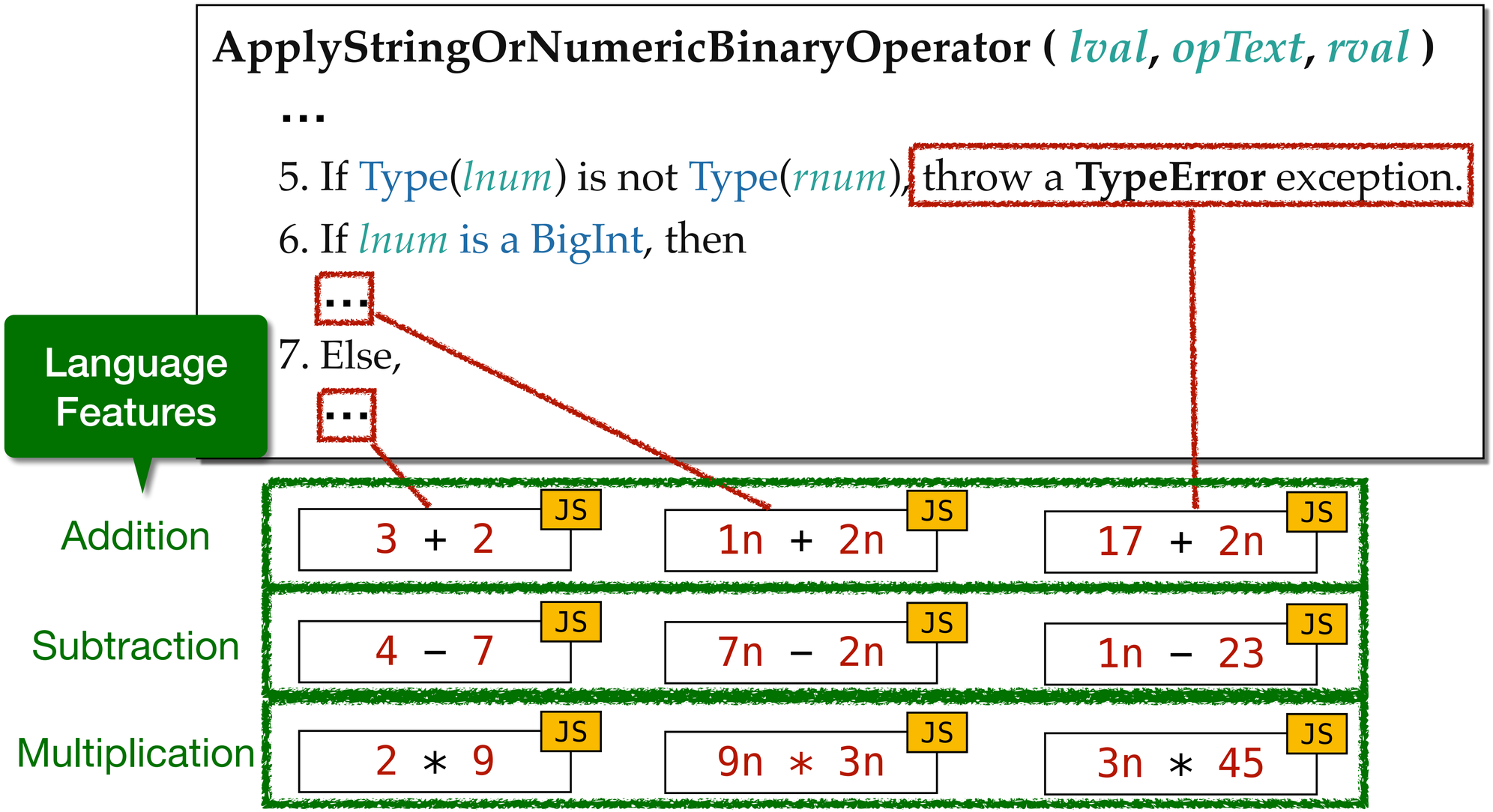
```
let x = 42;
assert(x == 42);
```

```
let x = ![];
assert(x == false);
```

Specification Coverage



Feature-Sensitive Coverage

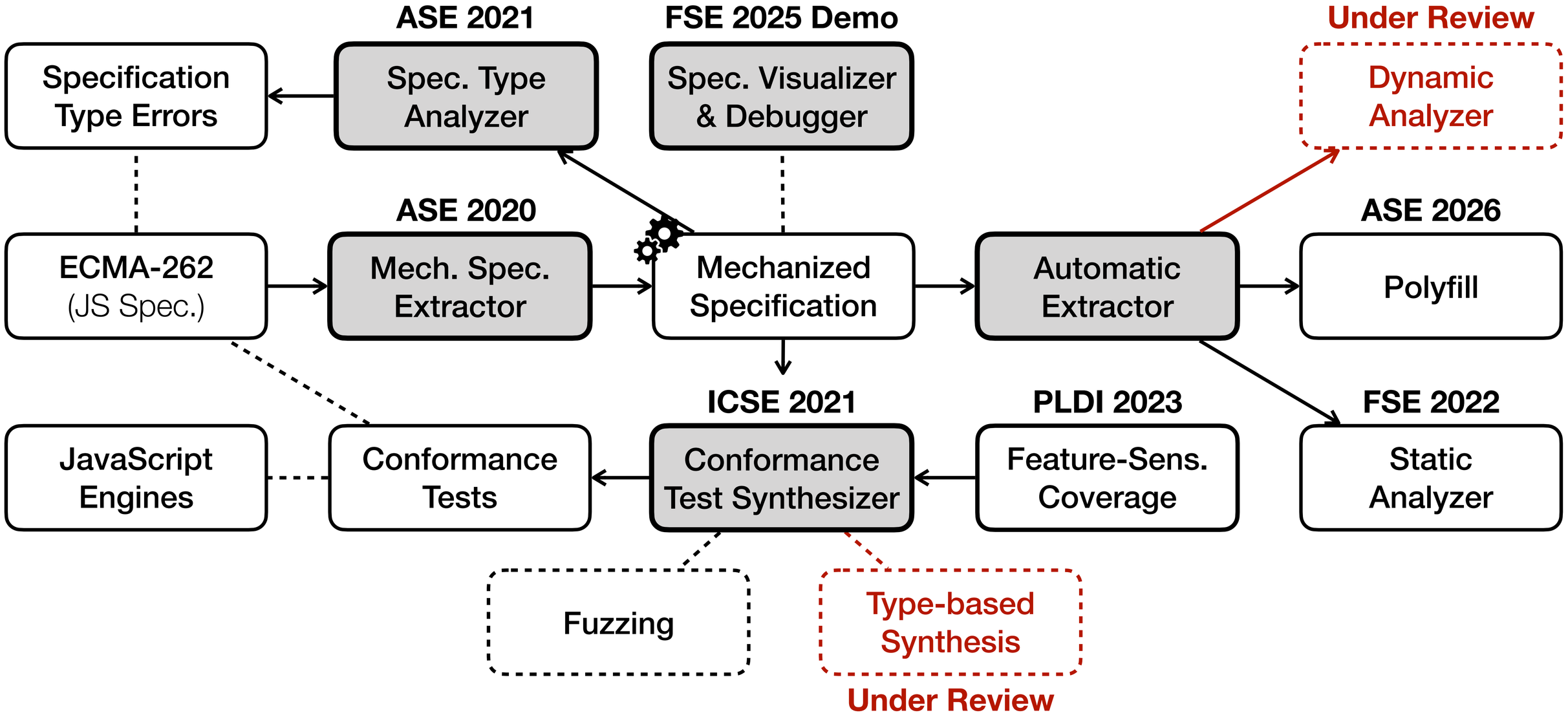


Specification Coverage

Kind	Name	Version	Release	# Detected Unique Bugs		
				# New	# Confirmed	# Reported
Engine	V8	v10.8.121	2022.10.06	3	3	4
	JSC	v615.1.10	2022.10.26	26	26	26
	GraalJS	v22.2.0	2022.07.26	11	11	11
	SpiderMonkey	v107.0b4	2022.10.24	2	4	4
	Total			**42**	**44**	**45**
Transpiler	Babel	v7.19.1	2022.09.15	37	37	39
	SWC	v1.3.10	2022.10.21	37	37	47
	Terser	v5.15.1	2022.10.05	19	19	19
	Obfuscator	v4.0.0	2022.02.15	1	2	7
	Total			**94**	**95**	**112**
Total				**136**	**139**	**157**

42 Bugs
In Engines

94 Bugs
In Transpilers


ASE 2021
FSE 2025 Demo
Under Review
Specification Type Errors
Spec. Type Analyzer
Spec. Visualizer & Debugger
Dynamic Analyzer
ASE 2020
ASE 2026
ECMA-262 (JS Spec.)
Mech. Spec. Extractor
Mechanized Specification
Automatic Extractor
Polyfill
ICSE 2021
PLDI 2023
FSE 2022
JavaScript Engines
Conformance Tests
Conformance Test Synthesizer
Feature-Sens. Coverage
Static Analyzer
Fuzzing
Type-based Synthesis
Under Review

Official tool used in CI system of ECMA-262 and Test262





https://github.com/es-meta/esmeta

